

ADDING LOCATION TRACKING To The Wearable Interface Device

ASHWINI KUMAR SINHA

Here's how you can add location tracking, without GPS, to the project for a smartwatch-like HID with keyboard and touchpad, which was published in March issue. Traditional GPS-based systems have limitations, especially indoors or in areas where signals are too weak or difficult to get due to obstructions.

This is a novel approach to location tracking using Wi-Fi SSID

signals, magnetic compass data, and the Indusboard, which is a compact IoT device with Wi-Fi capabilities and a magnetic compass sensor. By leveraging Wi-Fi signals and compass data, real-time location tracking can be achieved even in challenging environments. SSID stands for service set identifier, an important identifier for wireless networks that is assigned to a Wi-Fi network during router setup.

The SSID-based location tracking

system can be applied in various scenarios such as indoor navigation in buildings where GPS signals are weak or unavailable, asset tracking in warehouses or manufacturing facilities, monitoring and tracking personnel movement in large venues or events, and location-based advertising and marketing campaigns.

Here, the Indusboard (or an ESP32) is used to scan and connect to networks, extracting location data based on the network. It also utilises a magnetic compass to determine direction. Using the GC9A01 round display, the device shows the location data and connection states to the user wearing it.

The author's prototype is shown in Fig. 1. The components required for the project are listed in Bill of Materials table.

BILL OF MATERIALS

Components	Description	Quantity
Indusboard or ESP32	Development board	1
USB Type C	Adaptor	1
GC9A01 driver (MOD1)	Round touch display	1
CST816S touch sensor (CN1)	Touch sensor	1

Note: The Indusboard has a built-in 3D magnetic compass sensor for direction sensing. While using ESP32, an external compass sensor will need to be added.



Fig. 1: Author's prototype

Software

For coding, install the Arduino_GFX library, which helps drive the display. This library can be installed using the library manager found in the left bar of the new Arduino IDE.

Next, set the SPI display pins. On the board, any free pins can be configured; this is why the Indusboard has flexible software SPI capabilities. We have used pins 21, 1, 2, and 3 of the Indusboard for SPI, connecting them to the MOSI, SCK, CS, and D/C pins of the GC9A01 driver (MOD1), respectively.

Fig. 2 shows a snippet of the source code.

Circuit and working

Fig. 3 shows the circuit diagram for wearable location tracking using

```

1  #include <WiFi.h>
2  #include <HTTPClient.h>
3  #include <ArduinoJson.h>
4
5  const char *ssid = "IRONMAN";
6  const char *password = "1234567890";
7
8  #include <Arduino_GFX_Library.h>
9
10 Arduino_DataBus *bus = new Arduino_ESP32SPI( 3/* DC */, 2 /* CS */, 1/* SCK */, 21/* MOSI */, -1 /* MISO */);
11 Arduino_GFX *gfx = new Arduino_GC9A01(bus, 4 /* RST */, 3 /* rotation */, true /* IPS */);
12
13
14
15 /*****
16  * End of Arduino_GFX setting
17  *****/
18
19 /* more fonts at: https://github.com/moononournation/ArduinoFreeFontFile.git */
20 #include "FreeMono8pt7b.h"
21 #include "FreeSansBold10pt7b.h"
22 #include "FreeSerifBoldItalic12pt7b.h"
23
24 void setup(void)
25 {
26   Serial.begin(115200);

```

Fig. 2: Snippet of the code